

LAPORAN TUGAS II PEMROSESAN CITRA DIGITAL

Image Enhancement

Diajukan sebagai tugas Mata Kuliah IF4073 Pemrosesan Citra Digital



Disusun Oleh:
Haikal Assyauqi 13522052

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2025**

DAFTAR ISI

DAFTAR ISI.....	2
BAB I	
DESKRIPSI MASALAH.....	2
BAB II	
GRAPHICAL USER INTERFACE (GUI) PROGRAM.....	3
BAB III	
PENJELASAN KODE DAN PENGUJIAN PROGRAM.....	6
1. Screenshot Kode.....	6
2. Hasil Pengujian Program.....	23
BAB IV	
ANALISIS KODE.....	37
LAMPIRAN.....	43

BAB I

DESKRIPSI MASALAH

Pada tugas 2 dari mata kuliah IF4073 Pemrosesan Citra Digital, terdapat sejumlah program yang harus dibuat dengan menggunakan MATLAB antara lain sebagai berikut:

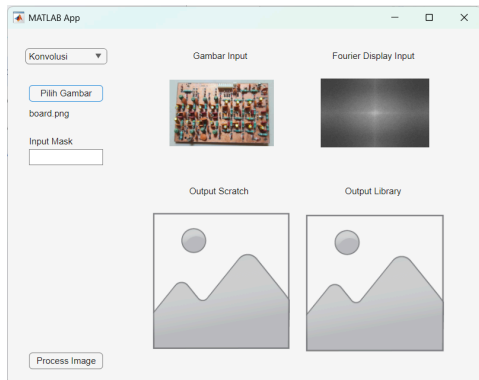
1. Melakukan konvolusi citra f berukuran sembarang ($M \times N$) dengan mask berukuran $n \times n$, baik pada citra grayscale maupun citra berwarna. Fungsi konvolusi tidak boleh menggunakan fungsi built-in dari Matlab (seperti `conv`), tetapi dibuat sendiri. Bandingkan hasil fungsi konvolusi buatan anda dengan fungsi konvolusi dari Matlab. Mekanisme untuk konvolusi pixel-pixel pinggir dibebaskan kepada anda. Contoh $n = 3, 5, 7$, dsb.
2. Membuat program Matlab untuk melakukan image smoothing (pada citra yang mengandung derau) atau blurring pada citra dalam: -
 - Ranah spasial menggunakan mean filter $n \times n$ dan gaussian filter $n \times n$. Gunakan fungsi konvolusi yang telah anda kerjakan pada poin 1. -
 - Ranah frekuensi menggunakan low-pass filter ILPF, GLPF, dan BLPF. Citra yang digunakan adalah citra grayscale dan citra berwarna. 3.
3. Membuat program Matlab untuk melakukan penapisan citra dalam ranah frekuensi menggunakan high-pass filter IHPF, GHPF, dan BHPF. Citra yang digunakan adalah citra grayscale dan citra berwarna.
4. Membuat program Matlab untuk melakukan penapisan citra dalam ranah frekuensi untuk menghasilkan citra yang lebih terang.
5. Membuat program Matlab untuk menambahkan derau salt and pepper pada citra (grayscale dan berwarna), lalu lakukan penghilangan derau dengan min filter, max filter, median filter, arithmetic mean filter, geometric filter, harmonic mean filter, contraharmonic mean filter, midpoint filter, alpha-trimmed mean filter. Tidak boleh menggunakan fungsi `medfilt2` di dalam Matlab.
6. Membuat program Matlab untuk menghilangkan derau periodik pada citra yang ada pada gambar
7. Membuat program Matlab untuk melakukan motion blurring pada citra (grayscale dan berwarna), lalu lakukan dekonvolusi pada citra tersebut dengan penapis Wiener. Program penapis Wiener anda buat sendiri (tidak boleh menggunakan fungsi Wiener di dalam Matlab)

BAB II

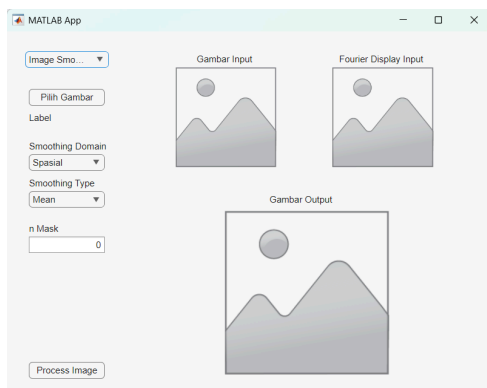
GRAPHICAL USER INTERFACE (GUI) PROGRAM

Berikut merupakan tampilan layar (screenshot) untuk GUI dari program yang dibuat dengan menggunakan MATLAB:

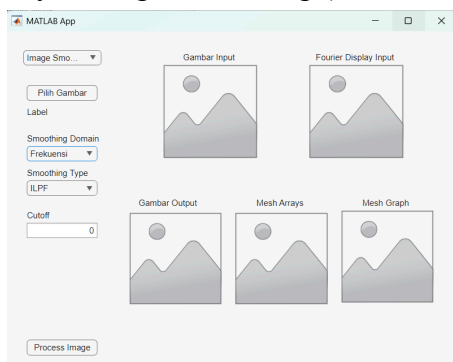
1. Layer Konvolusi



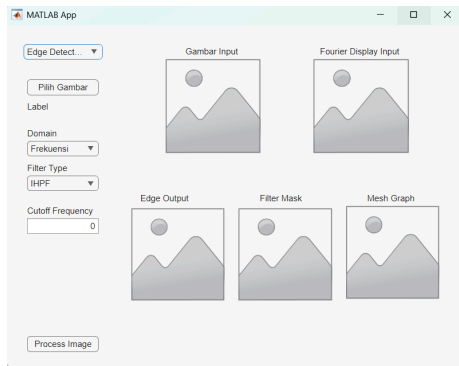
2. Layer Image Smoothing (Spatial)



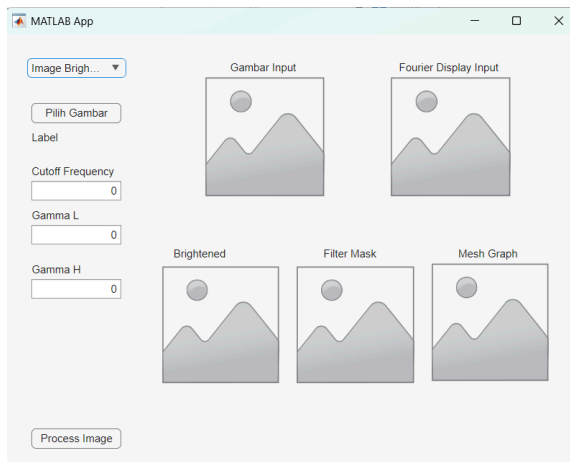
3. Layer Image Smoothing (Frekuensi)



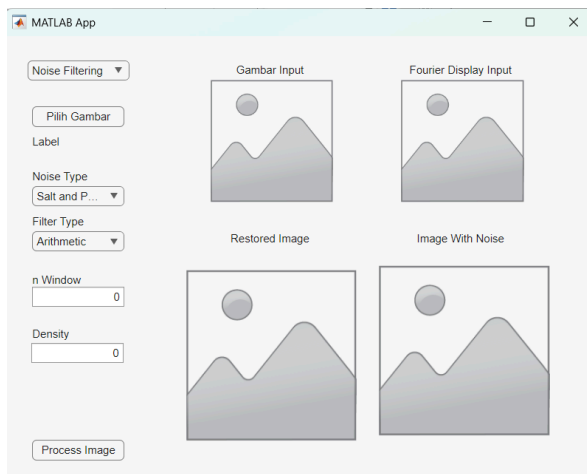
4. Layer Edge Detector (High Pass Filter)



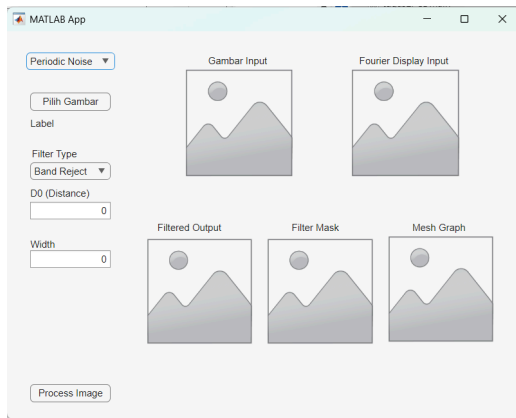
5. Layer Image Brightness (Homomorphic filter)



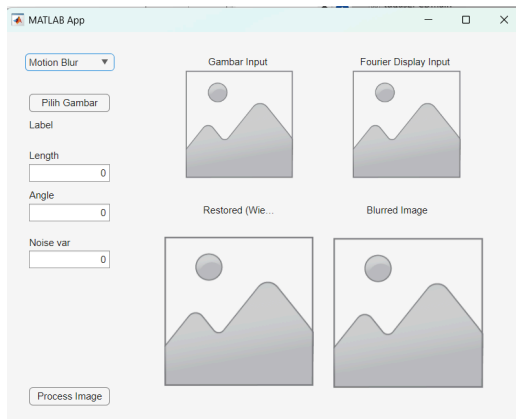
6. Layer NoiseFiltering



7. Layar Periodic Noise



8. Layar Motion Blur



BAB III

PENJELASAN KODE DAN PENGUJIAN PROGRAM

1. Screenshot Kode

a. Konvolusi

```
1 function output = convolution(imagePath, masking)
2     img = imread(imagePath);
3     img = im2double(img); % skala ke 0-1
4
5     % Normalisasi kernel (jika bukan deteksi tepi)
6     if sum(masking(:)) ~= 0
7         masking = masking / sum(masking(:));
8     end
9
10    [rows, cols, channels] = size(img);
11    [m, n] = size(masking);
12
13    offset_r = floor(m/2);
14    offset_c = floor(n/2);
15
16    output = zeros(rows, cols, channels);
17
18    for c = 1:channels
19        for i = (1 + offset_r):(rows - offset_r)
20            for j = (1 + offset_c):(cols - offset_c)
21                region = img(i-offset_r:i+offset_r, j-offset_c:j+offset_c, c);
22                val = sum(sum(region .* masking));
23                val = max(0, min(1, val));
24                output(i, j, c) = val;
25            end
26        end
27
28        % Salin border dari original
29        output(1:offset_r, :, c) = img(1:offset_r, :, c);
30        output(rows-offset_r+1:rows, :, c) = img(rows-offset_r+1:rows, :, c);
31        output(:, 1:offset_c, c) = img(:, 1:offset_c, c);
32        output(:, cols-offset_c+1:cols, c) = img(:, cols-offset_c+1:cols, c);
33    end
34
35    output = im2uint8(output); % balik ke uint8
36 end
37
```

b. Image Smoothing (Spatial)

```
1 function output = imageSmoothingSpatial(imagePath, nMask, sigma, type)
2     % Fungsi untuk melakukan image smoothing di ranah spasial
3     % imagePath: path ke file gambar
4     % nMask: ukuran kernel (n x n)
5     % sigma: standar deviasi untuk Gaussian filter
6     % type: jenis filter ("Mean" atau "Gaussian")
7
8     if strcmp(type, "Mean")
9         mask = ones(nMask, nMask) / (nMask * nMask);
10        output = convolusion(imagePath, mask);
11
12    elseif strcmp(type, "Gaussian")
13        mask = generateGaussianKernel(nMask, sigma);
14        output = convolusion(imagePath, mask);
15
16    else
17        error('Type harus "Mean" atau "Gaussian"');
18    end
19 end
20
21 function kernel = generateGaussianKernel(n, sigma)
22     center = floor(n/2);
23
24     kernel = zeros(n, n);
25
26     k = 1 / (2 * pi * sigma^2);
27
28     % Generate Gaussian kernel
29     for i = 1:n
30         for j = 1:n
31             % Hitung jarak dari center
32             x = i - center - 1;
33             y = j - center - 1;
34
35             % Formula Gaussian 2D
36             exponent = -(x^2 + y^2) / (2 * sigma^2);
37             kernel(i, j) = k * exp(exponent);
38         end
39     end
40 end
```

c. Image Smoothing (Frekuensi)

```
1 function [output, filterMask, filterImage] = imageSmoothingFrequency(imagePath, cutOff, order, type)
2 % Fungsi untuk melakukan image smoothing di ranah frekuensi
3 % imagePath : path ke file gambar
4 % cutOff : cutoff frequency D0 (radius)
5 % order : order untuk Butterworth (n), diabaikan untuk ILPF dan GLPF
6 % type : jenis filter ("ILPF", "GLPF", atau "BLPF")
7 %
8 % OUTPUT:
9 % output : hasil filtering (image)
10 % filterMask : matrix H(u,v) dari filter
11 % filterImage : image dari 3D mesh graph (untuk ditampilkan di uimage)
12
13 % Baca gambar
14 img = imread(imagePath);
15
16 % Simpan info untuk rekonstruksi
17 isColor = (size(img, 3) == 3);
18
19 % Dapatkan ukuran untuk membuat filter
20 if isColor
21     [M, N, ~] = size(img);
22 else
23     [M, N] = size(img);
24 end
25
26 % Buat filter mask (ini akan menjadi output ke-2)
27 filterMask = createFilter(M, N, cutOff, order, type);
28
29 % Jika RGB, process setiap channel
30 if isColor
31     % Process setiap channel secara terpisah
32     R = processChannel(double(img(:, :, 1)), filterMask);
33     G = processChannel(double(img(:, :, 2)), filterMask);
34     B = processChannel(double(img(:, :, 3)), filterMask);
35
36     % Gabungkan kembali
37     output = cat(3, R, G, B);
38 else
39     % Grayscale
40     output = processChannel(double(img), filterMask);
41 end
42
43 % Konversi ke uint8
44 output = uint8(output);
45
46 filterImage = captureFilterVisualization(filterMask, cutOff, type, order);
47 end
48
49 function filtered = processChannel(channel, H)
50 % Process satu channel image dengan filter H yang sudah dibuat
51 % channel : matrix 2D (M x N)
52 % H : filter mask (M x N)
53
54 F = fft2(channel);
55
56 F_shifted = fftshift(F);
57
58 G_shifted = F_shifted .* H;
59
60 G = ifftshift(G_shifted);
61
62 filtered = ifft2(G);
63
64 filtered = real(filtered);
65
66 filtered = max(0, min(255, filtered));
67 end
68
69 function H = createFilter(M, N, D0, n, type)
70 % Membuat filter mask H(u,v)
71 % M, N : ukuran image
72 % D0 : cutoff frequency
73 % n : order (untuk Butterworth)
74 % type : "ILPF", "GLPF", "BLPF"
75
76 % Buat koordinat mesh untuk u dan v
77 [u, v] = meshgrid(1:N, 1:M);
78
79 % Hitung center dari frequency domain
80 center_u = floor(N/2) + 1;
81 center_v = floor(M/2) + 1;
82
83 % Hitung jarak D(u,v) dari setiap point ke center
84 D = sqrt((u - center_u).^2 + (v - center_v).^2);
85
86 switch type
87     case 'ILPF'
88         % Ideal Low-Pass Filter
89         % H(u,v) = 1 if D(u,v) <= D0, else 0
90         H = double(D <= D0);
91
92     case 'GLPF'
93         % Gaussian Low-Pass Filter
94         % H(u,v) = exp(-(D^2(u,v)) / (2*D0^2))
95         H = exp(-(D.^2) / (2 * D0^2));
96
97     case 'BLPF'
98         % Butterworth Low-Pass Filter
99         % H(u,v) = 1 / (1 + (D(u,v)/D0)^(2n))
100
101         % Avoid division by zero
102         D(D == 0) = 0.0001;
103
104         % Butterworth formula
105         H = 1 ./ (1 + (D / D0).^(2*n));
106
107     otherwise
108         error('Type harus "ILPF", "GLPF", atau "BLPF"');
109 end
110 end
```

```

1 function filterImage = captureFilterVisualization(H, D0, type, order)
2     % Membuat visualisasi 3D mesh graph dari filter dan capture sebagai image
3     % H      : filter mask (M x N)
4     % D0     : cutoff frequency
5     % type   : tipe filter
6     % order  : order (untuk display title)
7     %
8     % OUTPUT:
9     % filterImage : uint8 RGB image dari visualization
10
11     figHandle = figure('Visible', 'off', 'Position', [0 0 800 600]);
12
13     [M, N] = size(H);
14
15     visualSize = min(200, min(M, N));
16     center_v = floor(M/2) + 1;
17     center_u = floor(N/2) + 1;
18
19     half_size = floor(visualSize/2);
20     row_start = max(1, center_v - half_size);
21     row_end = min(M, center_v + half_size);
22     col_start = max(1, center_u - half_size);
23     col_end = min(N, center_u + half_size);
24
25     H_subset = H(row_start:row_end, col_start:col_end);
26
27     % Buat koordinat untuk plotting
28     [rows, cols] = size(H_subset);
29     [X, Y] = meshgrid(1:cols, 1:rows);
30
31     mesh(X, Y, H_subset);
32
33     colormap(jet);
34     colorbar;
35
36     xlabel('u (Horizontal Frequency)');
37     ylabel('v (Vertical Frequency)');
38     zlabel('H(u,v) - Filter Response');
39
40     if strcmp(type, 'BLPF')
41         titleStr = sprintf('%s Filter (D_0=%d, n=%d)', type, round(D0), order);
42     else
43         titleStr = sprintf('%s Filter (D_0=%d)', type, round(D0));
44     end
45     title(titleStr);
46
47     view(-37.5, 30);
48
49     grid on;
50
51     zlim([0 1]);
52
53     lighting gouraud;
54     shading interp;
55
56     frame = getframe(figHandle);
57     filterImage = frame.cdata;
58
59     % Tutup figure
60     close(figHandle);
61 end

```

d. Edge Detector

```
1 function [output, filterMask, filterImage] = edgeDetection(imagePath, cutOff, order, type)
2 % Edge Detection menggunakan High-Pass Filter di frequency domain
3 % imagePath : path ke file gambar
4 % cutOff : cutoff frequency D0
5 % order : order untuk Butterworth (n)
6 % type : 'IHPF', 'GHPF', atau 'BHPF'
7
8 % Step 1: Baca dan konversi gambar
9 f = imread(imagePath);
10
11 % Konversi ke grayscale jika color
12 if size(f, 3) == 3
13     f = rgb2gray(f);
14 end
15
16 f = im2double(f);
17 [M, N] = size(f);
18
19 % Step 2: Tentukan parameter padding
20 P = 2 * M;
21 Q = 2 * N;
22
23 % Step 3: Bentuk citra padding
24 fp = zeros(P, Q);
25 fp(1:M, 1:N) = f;
26
27 % Step 4: Transformasi Fourier
28 F = fftshift(fft2(fp));
29
30 % Step 5: Bangkitkan High-Pass Filter H
31 H = createHighPassFilter(P, Q, cutOff, order, type);
32
33 % Step 6: Kalikan F dengan H
34 G = H .* F;
35 G1 = ifftshift(G);
36
37 % Step 7: Inverse FFT
38 G2 = real(ifft2(G1));
39
40 % Step 8: Potong kembali ke ukuran semula
41 output = G2(1:M, 1:N);
42
43 % Normalisasi output untuk display
44 output = normalizeEdgeImage(output);
45
46 % Generate filter mask untuk display
47 filterMask = mat2gray(H);
48 filterMask = uint8(filterMask * 255);
49
50 % Generate 3D visualization
51 filterImage = captureFilterVisualization(H, cutOff, type, order);
52 end
```

```

1 function H = createHighPassFilter(P, Q, D0, n, type)
2     % Membuat High-Pass Filter di frequency domain
3
4     % Set up range of variables
5     u = 0:(P-1);
6     v = 0:(Q-1);
7
8     % Compute the indices for use in meshgrid
9     idx = find(u > P/2);
10    u(idx) = u(idx) - P;
11    idy = find(v > Q/2);
12    v(idy) = v(idy) - Q;
13
14    % Compute the meshgrid arrays
15    [V, U] = meshgrid(v, u);
16
17    % Distance from center
18    D = sqrt(U.^2 + V.^2);
19
20    % Generate filter based on type
21    switch type
22        case 'IHPF' % Ideal High-Pass Filter
23            H = double(D > D0);
24
25        case 'GHPF' % Gaussian High-Pass Filter
26            H = 1 - exp(-(D.^2) / (2 * D0^2));
27
28        case 'BHPF' % Butterworth High-Pass Filter
29            D(D == 0) = eps; % Avoid division by zero
30            H = 1 ./ (1 + (D0 ./ D).^(2 * n));
31
32        otherwise
33            error('Type harus "IHPF", "GHPF", atau "BHPF"');
34    end
35
36    % Shift H untuk matching dengan fftshift
37    H = fftshift(H);
38 end

```



```

1 function output = normalizeEdgeImage(img)
2     % Normalisasi hasil edge detection
3
4     % Ambil absolute value
5     img = abs(img);
6
7     % Normalisasi ke [0, 1]
8     minVal = min(img(:));
9     maxVal = max(img(:));
10
11     if maxVal > minVal
12         img = (img - minVal) / (maxVal - minVal);
13     end
14
15     % Tingkatkan kontras dengan histogram stretching
16     % Gunakan percentile untuk robustness
17     low_in = prctile(img(:), 1);
18     high_in = prctile(img(:), 99);
19
20     if high_in > low_in
21         img = (img - low_in) / (high_in - low_in);
22         img = max(0, min(1, img));
23     end
24
25     % Gamma correction untuk highlight edges
26     img = img .^ 0.5;
27
28     % Convert to uint8
29     output = uint8(img * 255);
30 end
31
32 function filterImage = captureFilterVisualization(H, D0, type, order)
33     % Visualisasi 3D mesh dari filter
34
35     figHandle = figure('Visible', 'off', 'Position', [0 0 800 600]);
36
37     [P, Q] = size(H);
38
39     % Ambil subset untuk visualisasi (center region)
40     visualSize = min(200, min(P, Q));
41     center_p = floor(P/2) + 1;
42     center_q = floor(Q/2) + 1;
43
44     half_size = floor(visualSize/2);
45     row_start = max(1, center_p - half_size);
46     row_end = min(P, center_p + half_size);
47     col_start = max(1, center_q - half_size);
48     col_end = min(Q, center_q + half_size);
49
50     H_subset = H(row_start:row_end, col_start:col_end);
51
52     % Create mesh
53     [rows, cols] = size(H_subset);
54     [X, Y] = meshgrid(1:cols, 1:rows);
55
56     mesh(X, Y, H_subset);
57     colormap(jet);
58     colorbar;
59
60     xlabel('u (Horizontal Frequency)');
61     ylabel('v (Vertical Frequency)');
62     zlabel('H(u,v) - Filter Response');
63
64     % Title
65     if strcmp(type, 'BHPF')
66         titleStr = sprintf('%s HPF (D_0=%d, n=%d)', type, round(D0), order);
67     else
68         titleStr = sprintf('%s HPF (D_0=%d)', type, round(D0));
69     end
70     title(titleStr);
71
72     view(-37.5, 30);
73     grid on;
74     zlim([0 1]);
75     lighting gouraud;
76     shading interp;
77
78     % Capture figure
79     frame = getframe(figHandle);
80     filterImage = frame.cdata;
81
82     close(figHandle);
83 end

```

e. Image Brightness

```
1 function [output, filterMask, filterImage] = homomorphicFiltering(imagePath, cutOff, gammal, gammaH)
2     img = imread(imagePath);
3     isColor = (size(img, 3) == 3);
4
5     if isColor
6         [M, N, ~] = size(img);
7     else
8         [M, N] = size(img);
9     end
10
11     c = 1.0;
12     filterMask = createHomomorphicFilter(M, N, cutOff, gammal, gammaH, c);
13
14     if isColor
15         R = processChannelHomomorphic(double(img(:,:,1)), filterMask);
16         G = processChannelHomomorphic(double(img(:,:,2)), filterMask);
17         B = processChannelHomomorphic(double(img(:,:,3)), filterMask);
18         output = cat(3, R, G, B);
19     else
20         output = processChannelHomomorphic(double(img), filterMask);
21     end
22
23     p1 = prctile(output(:), 2);
24     p99 = prctile(output(:), 98);
25     output = (output - p1) / (p99 - p1);
26     output(output < 0) = 0;
27     output(output > 1) = 1;
28     output = output * 255;
29
30     output = uint8(output);
31
32     filterImage = captureHomomorphicVisualization(filterMask, cutOff, gammal, gammaH, c);
33 end
34
35 function filtered = processChannelHomomorphic(channel, H)
36     channel(channel < 1) = 1;
37
38     z = log(channel);
39
40     Z = fft2(z);
41     Z_shifted = fftshift(Z);
42
43     S_shifted = Z_shifted .* H;
44
45     S = ifftshift(S_shifted);
46     s = real(ifft2(S));
47
48     g = exp(s);
49
50     filtered = g;
51 end
52
53 function H = createHomomorphicFilter(M, N, D0, gammal, gammaH, c)
54     [u, v] = meshgrid(1:N, 1:M);
55     center_u = floor(N/2) + 1;
56     center_v = floor(M/2) + 1;
57
58     D = sqrt((u - center_u).^2 + (v - center_v).^2);
59
60     H = (gammaH - gammal) * (1 - exp(-c * (D.^2) / (D0.^2))) + gammal;
61 end
62
63 function filterImage = captureHomomorphicVisualization(H, D0, gammal, gammaH, c)
64     figHandle = figure('Visible', 'off', 'Position', [0 0 800 600]);
65
66     [M, N] = size(H);
67     visualSize = min(200, min(M, N));
68     center_v = floor(M/2) + 1;
69     center_u = floor(N/2) + 1;
70     half_size = floor(visualSize/2);
71
72     row_start = max(1, center_v - half_size);
73     row_end = min(M, center_v + half_size);
74     col_start = max(1, center_u - half_size);
75     col_end = min(N, center_u + half_size);
76
77     H_subset = H(row_start:row_end, col_start:col_end);
78     [rows, cols] = size(H_subset);
79     [X, Y] = meshgrid(1:cols, 1:rows);
80
81     mesh(X, Y, H_subset);
82     colormap(jet);
83     colorbar;
84     xlabel('u (Horizontal Frequency)');
85     ylabel('v (Vertical Frequency)');
86     zlabel('H(u,v)');
87     titleStr = sprintf('Homomorphic Filter (D0=%d) \gammaL=%X.2f \gammaH=%X.2f', D0, gammal, gammaH);
88     title(titleStr);
89     view(-37.5, 30);
90     grid on;
91     lighting gouraud;
92     shading interp;
93
94     frame = getframe(figHandle);
95     filterImage = frame.cdata;
96     close(figHandle);
97 end
```

f. Noise Filtering

```
1 function [output, pictWithNoise] = noiseRestoration(imagePath, noiseType, filterType, nWindow, mean, variance, density)
2 % Fungsi untuk noise restoration dengan berbagai filter
3 % imagePath : path ke file gambar
4 % noiseType : 'salt_pepper' atau 'gaussian'
5 % filterType : 'min', 'max', 'median', 'arithmetic', 'geometric',
6 %             'harmonic', 'contraharmonic', 'midpoint', 'alpha_trimmed'
7 % nWindow : ukuran window filter (3, 5, 7, dll - harus ganjil)
8 % mean : mean untuk gaussian noise (biasanya 0)
9 % variance : variance untuk gaussian noise (misal 0.01)
10 % density : density untuk salt & pepper (misal 0.05 = 5%)
11 %
12 % OUTPUT:
13 % output : hasil filtering (restored image)
14 % pictWithNoise : gambar dengan noise
15 %
16 % Baca gambar
17 img = imread(imagePath);
18 isColor = (size(img, 3) == 3);
19
20 % Tambahkan noise
21 switch noiseType
22     case 'salt_pepper'
23         noisyImg = addSaltPepperNoise(img, density);
24     case 'gaussian'
25         noisyImg = addGaussianNoise(img, mean, variance);
26     otherwise
27         error('noiseType harus "salt_pepper" atau "gaussian"');
28 end
29
30 pictWithNoise = noisyImg;
31
32 % Apply filter untuk menghilangkan noise
33 if isColor
34     % Proses setiap channel
35     R = applyFilter(double(noisyImg(:,1)), filterType, nWindow);
36     G = applyFilter(double(noisyImg(:,2)), filterType, nWindow);
37     B = applyFilter(double(noisyImg(:,3)), filterType, nWindow);
38     output = cat(3, R, G, B);
39 else
40     output = applyFilter(double(noisyImg), filterType, nWindow);
41 end
42
43 output = uint8(output);
44 end
45
46 %% ===== NOISE ADDITION =====
47
48 function noisyImg = addSaltPepperNoise(img, density)
49 % Menambahkan salt & pepper noise
50 % density: probabilitas total noise (misal 0.05 = 5%)
51 % Salt (white) = density/2, Pepper (black) = density/2
52
53 [M, N, C] = size(img);
54 noisyImg = img;
55
56 % Salt density dan Pepper density
57 saltDensity = density / 2;
58 pepperDensity = density / 2;
59
60 % Generate random matrix
61 randMatrix = rand(M, N);
62
63 for c = 1:C
64     channel = noisyImg(:,c);
65
66     % Add pepper (black pixels = 0)
67     channel(randMatrix < pepperDensity) = 0;
68
69     % Add salt (white pixels = 255)
70     channel(randMatrix > (1 - saltDensity)) = 255;
71
72     noisyImg(:,c) = channel;
73 end
74 end
75
76 function noisyImg = addGaussianNoise(img, mean, variance)
77 % Menambahkan gaussian noise
78 % mean: rata-rata noise (biasanya 0)
79 % variance: variance noise (misal 0.01)
80
81 noisyImg = double(img);
82 [M, N, C] = size(img);
83
84 for c = 1:C
85     % Generate gaussian noise
86     noise = mean + sqrt(variance) * randn(M, N) * 255;
87
88     % Add noise to channel
89     channel = noisyImg(:,c) + noise;
90
91     % Clip to [0, 255]
92     channel = max(0, min(255, channel));
93
94     noisyImg(:,c) = channel;
95 end
96
97 noisyImg = uint8(noisyImg);
98 end
99
```

```

1 function filtered = applyFilter(img, filterType, nWindow)
2     % Apply filter ke image
3     % img: channel tunggal (grayscale atau satu channel RGB)
4     % filterType: jenis filter
5     % nWindow: ukuran window (harus ganjil)
6
7     if mod(nWindow, 2) == 0
8         error('nWindow harus ganjil (3, 5, 7, ...)');
9     end
10
11     [M, N] = size(img);
12     filtered = zeros(M, N);
13     pad = floor(nWindow / 2);
14
15     % Padding image (replicate)
16     imgPadded = padarray(img, [pad pad], 'replicate');
17
18     % Apply filter ke setiap pixel
19     for i = 1:M
20         for j = 1:N
21             % Extract window
22             window = imgPadded(i:i+nWindow-1, j:j+nWindow-1);
23
24             % Apply filter
25             switch filterType
26                 case 'min'
27                     filtered(i,j) = minFilter(window);
28                 case 'max'
29                     filtered(i,j) = maxFilter(window);
30                 case 'median'
31                     filtered(i,j) = medianFilter(window);
32                 case 'arithmetic'
33                     filtered(i,j) = arithmeticMeanFilter(window);
34                 case 'geomethric'
35                     filtered(i,j) = geometricMeanFilter(window);
36                 case 'harmonic'
37                     filtered(i,j) = harmonicMeanFilter(window);
38                 case 'contraharmonic'
39                     % Q parameter untuk contraharmonic (default = 1.5)
40                     Q = 1.5;
41                     filtered(i,j) = contraharmonicMeanFilter(window, Q);
42                 case 'midpoint'
43                     filtered(i,j) = midpointFilter(window);
44                 case 'alpha_trimmed'
45                     % d parameter untuk alpha-trimmed (delete d/2 from each end)
46                     d = 4; % For 5x5, delete 2 lowest and 2 highest
47                     filtered(i,j) = alphaTrimmedMeanFilter(window, d);
48                 otherwise
49                     error('filterType tidak dikenal');
50             end
51         end
52     end
53
54     % Clip to [0, 255]
55     filtered = max(0, min(255, filtered));
56 end

```

```

1 function val = minFilter(window)
2     % Min Filter - ambil nilai minimum
3     val = min(window(:));
4 end
5
6 function val = maxFilter(window)
7     % Max Filter - ambil nilai maximum
8     val = max(window(:));
9 end
10
11 function val = medianFilter(window)
12     values = sort(window(:));
13     n = length(values);
14
15     if mod(n, 2) == 1
16         % Odd number of elements
17         val = values((n+1)/2);
18     else
19         % Even number of elements
20         val = (values(n/2) + values(n/2 + 1)) / 2;
21     end
22 end
23
24 function val = arithmeticMeanFilter(window)
25     % Arithmetic Mean Filter
26     val = mean(window(:));
27 end
28
29 function val = geometricMeanFilter(window)
30     % Geometric Mean Filter
31     % G = (prod(pixels))^(1/n)
32     pixels = window(:);
33
34     % Avoid log(0)
35     pixels(pixels == 0) = 1;
36
37     % Geometric mean using log
38     val = exp(mean(log(pixels)));
39 end
40
41 function val = harmonicMeanFilter(window)
42     % Harmonic Mean Filter
43     % H = n / sum(1/pixels)
44     pixels = window(:);
45
46     % Avoid division by zero
47     pixels(pixels == 0) = 1;
48
49     n = length(pixels);
50     val = n / sum(1 ./ pixels);
51 end
52
53 function val = contraharmonicMeanFilter(window, Q)
54     % Contraharmonic Mean Filter
55     % C = sum(pixels^(Q+1)) / sum(pixels^Q)
56     % Q > 0: eliminates pepper noise
57     % Q < 0: eliminates salt noise
58     pixels = window(:);
59
60     % Avoid issues with zero
61     pixels(pixels == 0) = 1;
62
63     numerator = sum(pixels .^ (Q + 1));
64     denominator = sum(pixels .^ Q);
65
66     if denominator == 0
67         val = 0;
68     else
69         val = numerator / denominator;
70     end
71 end
72
73 function val = midpointFilter(window)
74     % Midpoint Filter
75     % M = (min + max) / 2
76     val = (min(window(:)) + max(window(:))) / 2;
77 end
78
79 function val = alphaTrimmedMeanFilter(window, d)
80     % Alpha-Trimmed Mean Filter
81     % Buang d/2 nilai terkecil dan d/2 nilai terbesar, lalu hitung mean
82     values = sort(window(:));
83     n = length(values);
84
85     % Ensure d is valid
86     if d >= n
87         d = n - 1;
88     end
89
90     % Trim d/2 from each end
91     trimAmount = floor(d / 2);
92
93     if trimAmount > 0
94         trimmedValues = values(trimAmount+1 : end-trimAmount);
95     else
96         trimmedValues = values;
97     end
98
99     val = mean(trimmedValues);
100 end

```

g. Periodic Noise

```
1 function [output, filterMask, filterImage] = periodicNoise(imagePath, filterType, lowCutoff, highCutoff, distance, width)
2 % Fungsi untuk menghilangkan periodic noise menggunakan frequency domain filtering
3 % imagePath : path ke file gambar
4 % filterType : 'bandreject', 'bandpass', 'notch'
5 %
6 % PARAMETER UNTUK SETIAP FILTER:
7 % - BANDREJECT: distance, width (menggunakan D0=distance, W=width)
8 % - BANDPASS : lowCutoff, highCutoff
9 % - NOTCH : lowCutoff (sebagai jarak notch dari center)
10 %
11 % OUTPUT:
12 % output : hasil filtering (cleaned image)
13 % filterMask : matrix H(u,v) dari filter
14 % filterImage : visualisasi 3D mesh graph
15
16 % Order fixed = 2 untuk semua filter
17 n = 1;
18
19 % Baca gambar
20 img = imread(imagePath);
21
22 % Simpan info untuk rekonstruksi
23 isColor = (size(img, 3) == 3);
24
25 % Dapatkan ukuran untuk membuat filter
26 if isColor
27     [M, N, ~] = size(img);
28 else
29     [M, N] = size(img);
30 end
31
32 % Buat filter mask berdasarkan tipe
33 switch filterType
34     case 'bandreject'
35         % Bandreject: menggunakan distance (D0) dan width (W)
36         filterMask = createBandrejectFilter(M, N, distance, width, n);
37
38     case 'bandpass'
39         % Bandpass: menggunakan lowCutoff dan highCutoff
40         filterMask = createBandpassFilter(M, N, lowCutoff, highCutoff, n);
41
42     case 'notch'
43         % Notch: menggunakan lowCutoff sebagai notch position
44         filterMask = createNotchFilter(M, N, lowCutoff, n);
45
46     otherwise
47         error('filterType harus "bandreject", "bandpass", atau "notch"');
48 end
49
50 % Jika RGB, process setiap channel
51 if isColor
52     R = processChannelPeriodic(double(img(:,:,1)), filterMask);
53     G = processChannelPeriodic(double(img(:,:,2)), filterMask);
54     B = processChannelPeriodic(double(img(:,:,3)), filterMask);
55     output = cat(3, R, G, B);
56 else
57     output = processChannelPeriodic(double(img), filterMask);
58 end
59
60 % Konversi ke uint8
61 output = uint8(output);
62
63 % Buat visualisasi 3D mesh graph
64 filterImage = capturePeriodicNoiseVisualization(filterMask, filterType, ...
65     lowCutoff, highCutoff, distance, width, n);
66 end
67
68 function filtered = processChannelPeriodic(channel, H)
69     F = fft2(channel);
70     F_shifted = fftshift(F);
71     G_shifted = F_shifted .* H;
72     G = ifftshift(G_shifted);
73     filtered = ifft2(G);
74     filtered = real(filtered);
75
76     minVal = min(filtered(:));
77     maxVal = max(filtered(:));
78
79     if maxVal > minVal
80         filtered = (filtered - minVal) / (maxVal - minVal);
81         filtered = filtered.^ 0.7;
82         filtered = filtered * 255;
83     else
84         filtered = filtered - minVal;
85     end
86
87     filtered = max(0, min(255, filtered));
88 end
```

```

1 function H = createBandrejectFilter(M, N, D0, W, n)
2     % Membuat Butterworth Bandreject Filter
3     % M, N : ukuran image
4     % D0 : center frequency (distance from origin)
5     % W : bandwidth (width of rejected band)
6     % n : order (fixed = 2)
7     %
8     % Rejects frequencies in band around D0 with width W
9     %  $H(u,v) = 1 / (1 + [D(u,v)*W / (D^2(u,v) - D0^2)]^{(2n)})$ 
10
11     % Buat koordinat mesh
12     [u, v] = meshgrid(1:N, 1:M);
13
14     % Hitung center
15     center_u = floor(N/2) + 1;
16     center_v = floor(M/2) + 1;
17
18     % Hitung jarak D(u,v) dari center
19     D = sqrt((u - center_u).^2 + (v - center_v).^2);
20
21     % Avoid division by zero
22     D(D == 0) = 0.0001;
23
24     % Butterworth Bandreject formula
25     numerator = D * W;
26     denominator = D.^2 - D0^2;
27
28     % Handle denominator = 0 (at exactly D0)
29     denominator(abs(denominator) < 0.0001) = 0.0001;
30
31     H = 1 ./ (1 + (numerator ./ denominator).^(2*n));
32 end
33
34 %% ===== BANDPASS FILTER =====
35
36 function H = createBandpassFilter(M, N, D0, D1, n)
37     if D1 <= D0
38         error('highCutoff (D1) harus > lowCutoff (D0)');
39     end
40
41     [u, v] = meshgrid(1:N, 1:M);
42     center_u = floor(N/2) + 1;
43     center_v = floor(M/2) + 1;
44     D = sqrt((u - center_u).^2 + (v - center_v).^2);
45
46     H_low = exp(-(D.^2) ./ (2 * D1^2));
47     H_high = exp(-(D.^2) ./ (2 * D0^2));
48
49     H = H_low .* (1 - H_high);
50
51     H = max(0, min(1, H));
52 end

```

```

1 function H = createNotchFilter(M, N, D_notch, n)
2     [u, v] = meshgrid(1:N, 1:M);
3     center_u = floor(N/2) + 1;
4     center_v = floor(M/2) + 1;
5     H = ones(M, N);
6
7     W_notch = max(10, D_notch * 0.3);
8
9     notch_u1 = center_u + D_notch;
10    notch_v1 = center_v + D_notch;
11
12    notch_u2 = center_u - D_notch;
13    notch_v2 = center_v - D_notch;
14
15    D1 = sqrt((u - notch_u1).^2 + (v - notch_v1).^2);
16    D2 = sqrt((u - notch_u2).^2 + (v - notch_v2).^2);
17
18    D1(D1 < 0.01) = 0.01;
19    D2(D2 < 0.01) = 0.01;
20
21    H_notch1 = 1 ./ (1 + (W_notch ./ D1).^(2*n));
22    H_notch2 = 1 ./ (1 + (W_notch ./ D2).^(2*n));
23
24    H = H .* H_notch1 .* H_notch2;
25 end
26
27
28 %% ===== VISUALIZATION =====
29
30 function filterImage = capturePeriodicNoiseVisualization(H, filterType, D0, D1, distance, W, n)
31     % Membuat visualisasi 3D mesh graph dari filter
32
33     % Buat figure invisible
34     figHandle = figure('Visible', 'off', 'Position', [0 0 800 600]);
35
36     % Ambil ukuran filter
37     [M, N] = size(H);
38
39     % Ambil subset untuk visualisasi
40     visualSize = min(200, min(M, N));
41     center_v = floor(M/2) + 1;
42     center_u = floor(N/2) + 1;
43
44     half_size = floor(visualSize/2);
45     row_start = max(1, center_v - half_size);
46     row_end = min(M, center_v + half_size);
47     col_start = max(1, center_u - half_size);
48     col_end = min(N, center_u + half_size);
49
50     H_subset = H(row_start:row_end, col_start:col_end);
51
52     % Buat koordinat untuk plotting
53     [rows, cols] = size(H_subset);
54     [X, Y] = meshgrid(1:cols, 1:rows);
55
56     % Plot 3D mesh
57     mesh(X, Y, H_subset);
58
59     % Atur properti grafik
60     colormap(jet);
61     colorbar;
62
63     % Labels dan title
64     xlabel('u (Horizontal Frequency)');
65     ylabel('v (Vertical Frequency)');
66     zlabel('H(u,v) - Filter Response');
67
68     % Title berdasarkan tipe
69     switch filterType
70         case 'bandreject'
71             titleStr = sprintf('Bandreject Filter\n(D_0=%d, W=%d, n=%d)', ...
72                 round(distance), round(W), n);
73         case 'bandpass'
74             titleStr = sprintf('Bandpass Filter\n(D_0=%d, D_1=%d, n=%d)', ...
75                 round(D0), round(D1), n);
76         case 'notch'
77             titleStr = sprintf('Notch Filter\n(D_(notch)=%d, n=%d)', ...
78                 round(D0), n);
79     end
80     title(titleStr);
81
82     % Viewing angle
83     view(-37.5, 30);
84
85     % Grid
86     grid on;
87
88     % Axis properties
89     zlim([0 1]);
90
91     % Lighting
92     lighting gouraud;
93     shading interp;
94
95     % Capture
96     frame = getframe(figHandle);
97     filterImage = frame.cdata;
98
99     % Close
100    close(figHandle);
101 end

```


h. Motion Blur

```
1 function [outputImage, imageWithMotionBlur] = motionBlur(imagePath, len, angle, noiseVar)
2 % MOTION BLUR RGB + WIENER RESTORATION TANPA FUNGSI FILT APAPUN
3 % -----
4 % imagePath : path file gambar
5 % len       : panjang motion blur
6 % angle     : sudut blur (0 = horizontal)
7 % noiseVar  : variansi noise (misal 0.0001)
8 %
9 % outputImage : hasil restorasi (Wiener)
10 % imageWithMotionBlur : gambar blur dengan noise
11
12 % ==== BACA GAMBAR ====
13 img = imread(imagePath);
14 img = double(img);
15 [M, N, C] = size(img);
16
17 % ==== BUAT PSF (tanpa fungsi bawaan) ====
18 PSF = createMotionBlurPSF(len, angle);
19 [psfM, psfN] = size(PSF);
20
21 % PAD PSF KE UKURAN GAMBAR
22 PSF_padded = zeros(M, N);
23 startM = floor((M - psfM) / 2) + 1;
24 startN = floor((N - psfN) / 2) + 1;
25 PSF_padded(startM:startM+psfM-1, startN:startN+psfN-1) = PSF;
26 H = fft2(fftshift(PSF_padded));
27
28 % ==== BLUR + NOISE + RESTORE PER CHANNEL ====
29 blurred = zeros(M, N, C);
30 restored = zeros(M, N, C);
31
32 for ch = 1:C
33     F = fft2(img(:,:,ch));
34     G = F .* H;
35     b = real(ifft2(G));
36
37     % Tambah noise Gaussian (jika ada)
38     if noiseVar > 0
39         noise = sqrt(noiseVar) * randn(M, N) * 255;
40         b = b + noise;
41     end
42
43     blurred(:,:,ch) = b;
44
45     % Wiener restoration
46     restoredFFT = wienerFilter(fft2(b), H, noiseVar, mean(img(:,:,ch), 'all'));
47     restored(:,:,ch) = real(ifft2(restoredFFT));
48 end
49
50 % ==== NORMALISASI DAN KONVERSI KE UINT8 ====
51 blurred = mat2gray(blurred) * 255;
52 restored = mat2gray(restored) * 255;
53
54 imageWithMotionBlur = uint8(blurred);
55 outputImage = uint8(restored);
56 end
57
58 % =====
59 function PSF = createMotionBlurPSF(len, theta)
60 % Membuat motion blur PSF manual tanpa fungsi filt apapun
61 len = max(1, round(len));
62 theta = mod(theta, 180);
63 theta_rad = theta * pi / 180;
64
65 psfSize = 2 * len + 1;
66 PSF = zeros(psfSize, psfSize);
67 center = ceil(psfSize / 2);
68
69 % Gambar garis lurus sesuai sudut
70 for i = -len:len
71     x = round(center + i * cos(theta_rad));
72     y = round(center - i * sin(theta_rad));
73     if x >= 1 && x <= psfSize && y >= 1 && y <= psfSize
74         PSF(y, x) = 1;
75     end
76 end
77
78 % Lakukan sedikit smoothing manual (gaussian kecil buatan)
79 % agar tidak terlalu tajam -> mencegah ringing
80 PSF = smooth2D(PSF);
81 PSF = PSF / sum(PSF(:));
82 end
```

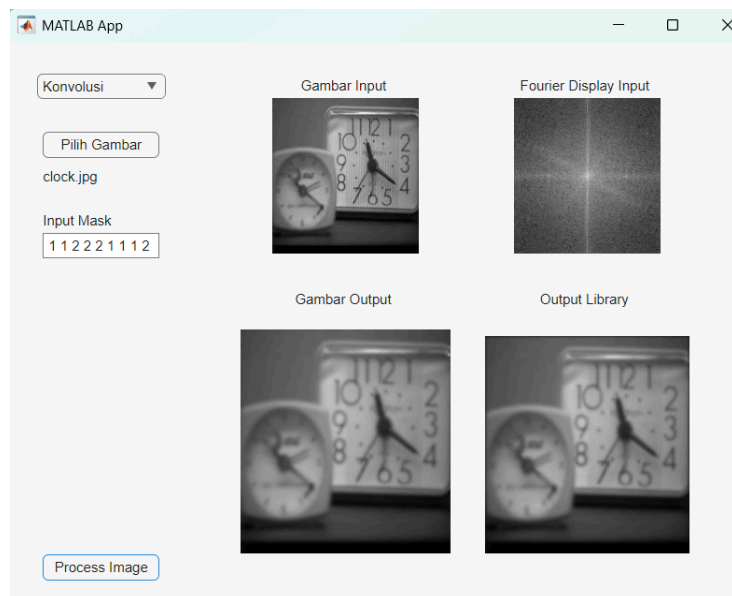
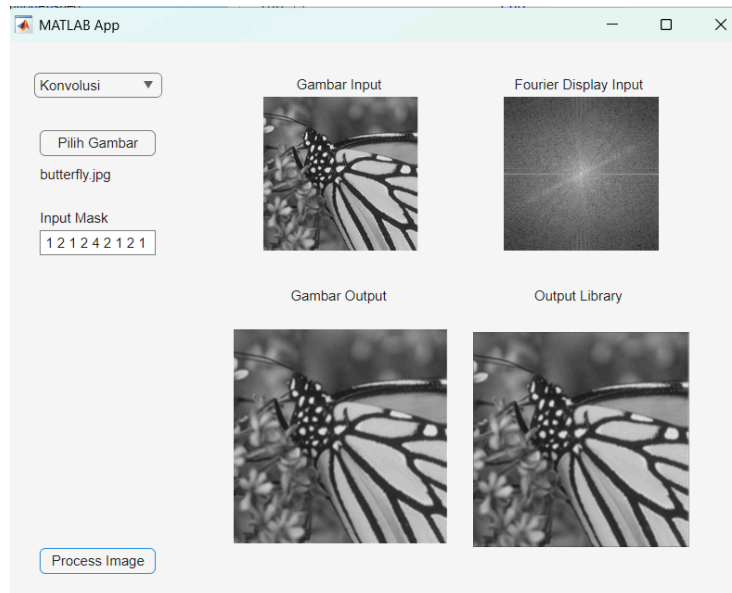


```
1 function F_restored = wienerFilter(G, H, noiseVar, signalPower)
2     K = 0.01 + noiseVar / (signalPower + eps);
3     H_conj = conj(H);
4     H_abs_sq = abs(H).^2;
5     denominator = H_abs_sq + K;
6     denominator(abs(denominator) < eps) = eps;
7     F_restored = (H_conj ./ denominator) .* G;
8 end
9
10 % =====
11 function A_smooth = smooth2D(A)
12     % Smooth manual pakai kernel Gaussian kecil tanpa fungsi filt
13     kernel = [1 2 1; 2 4 2; 1 2 1];
14     kernel = kernel / sum(kernel(:));
15     [m, n] = size(A);
16     A_smooth = zeros(m, n);
17
18     for i = 2:m-1
19         for j = 2:n-1
20             window = A(i-1:i+1, j-1:j+1);
21             A_smooth(i,j) = sum(sum(window .* kernel));
22         end
23     end
24 end
```

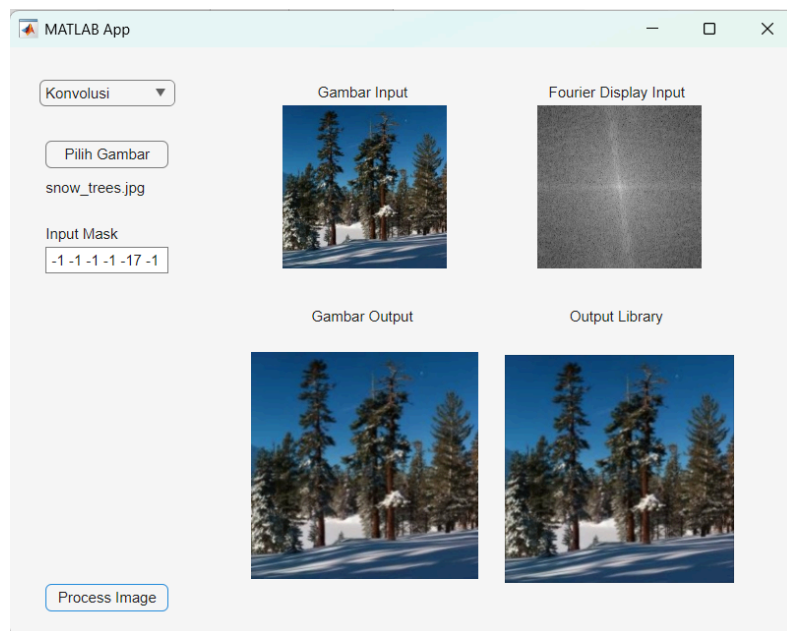
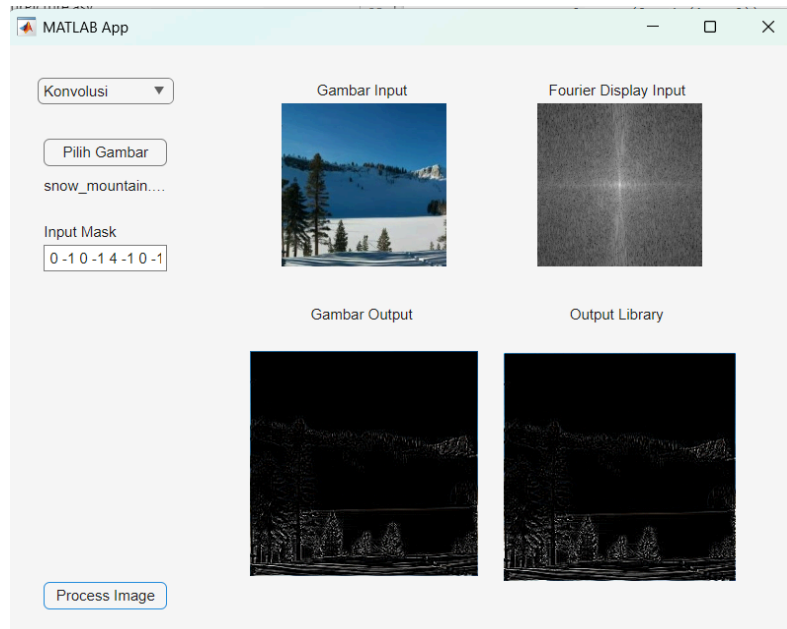
2. Hasil Pengujian Program

a. Konvolusi

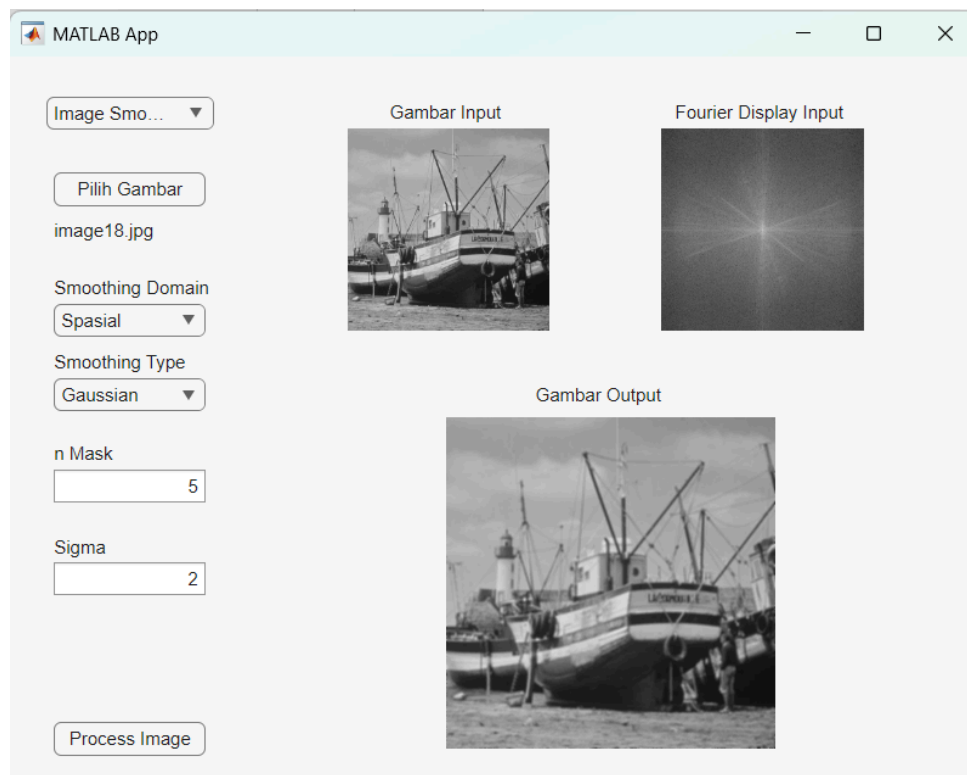
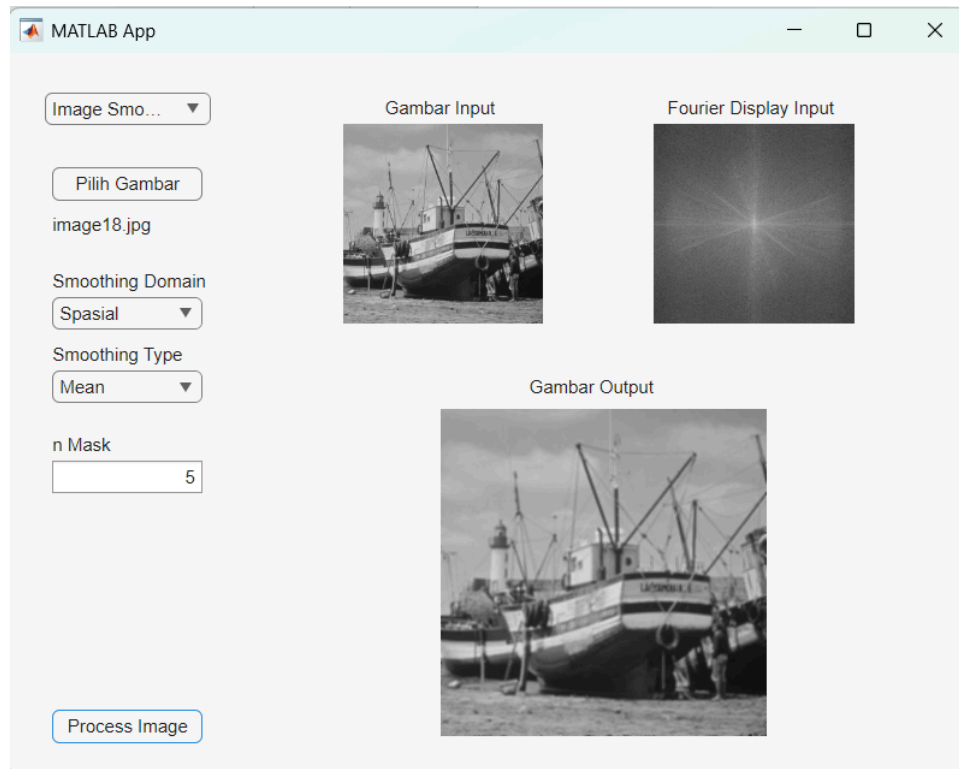
I. Gambar Grayscale



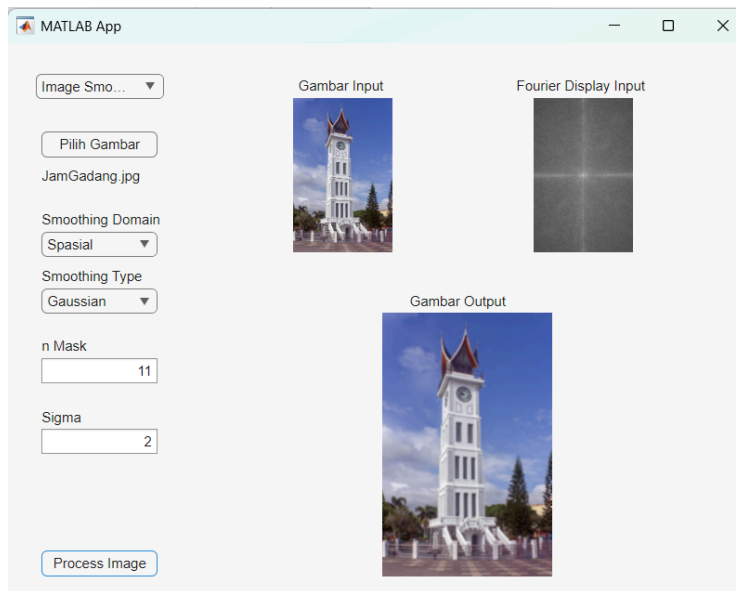
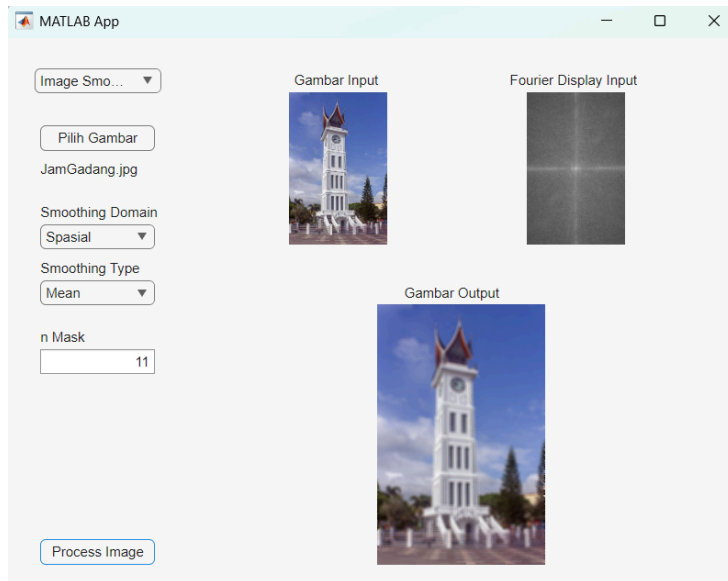
II. Gambar Berwarna



b. Image Smoothing (Partial)
I. Gambar Grayscale

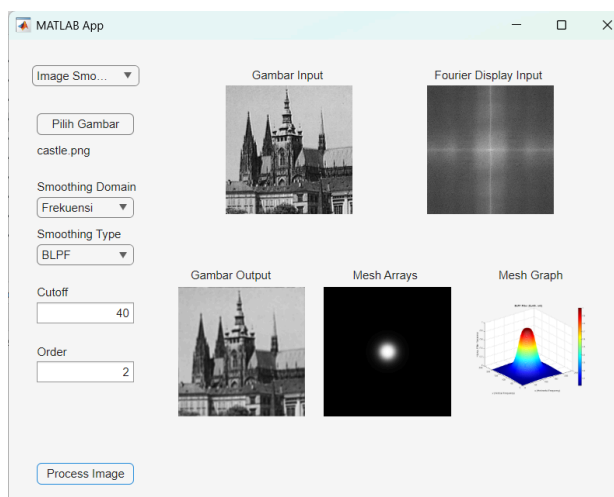
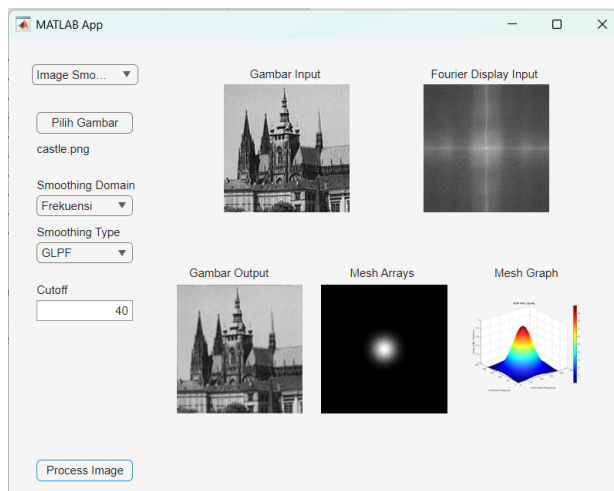
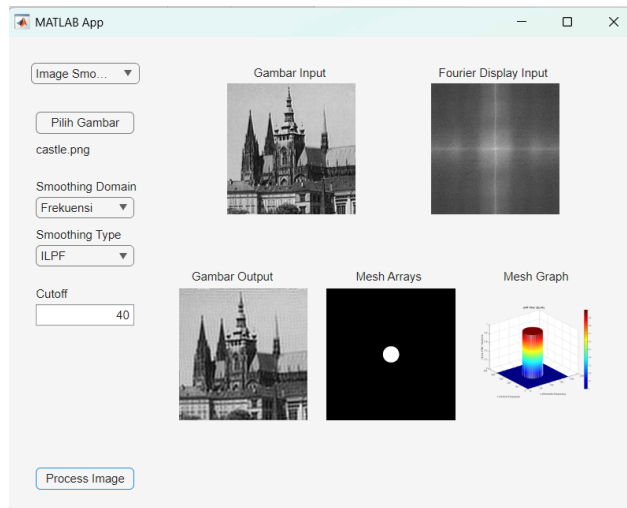


II. Gambar Berwarna

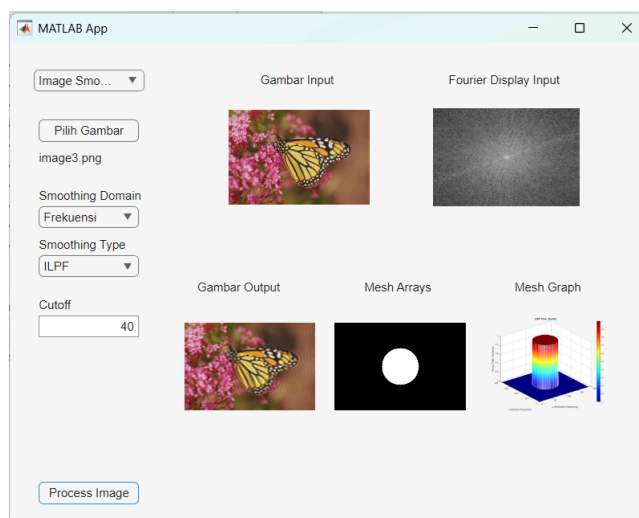
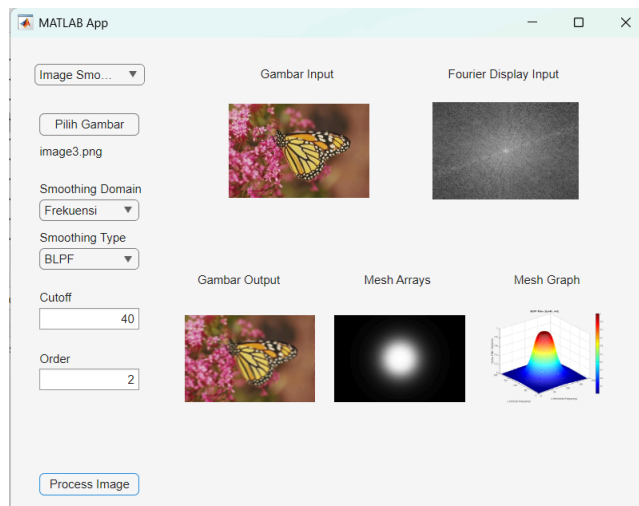
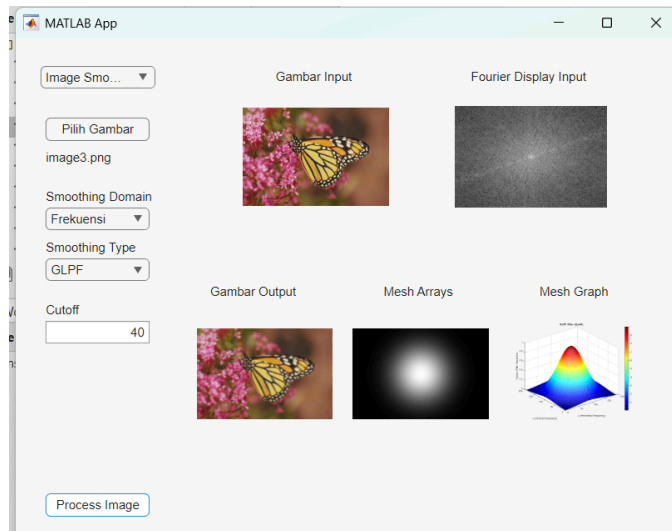


c. Image Smoothing (Frekuensi)

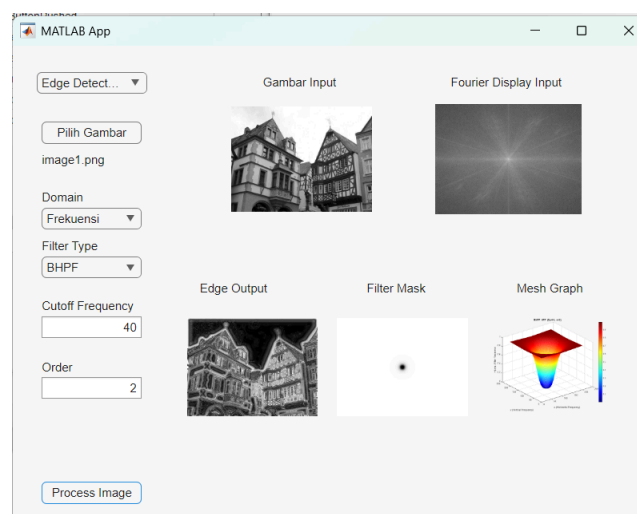
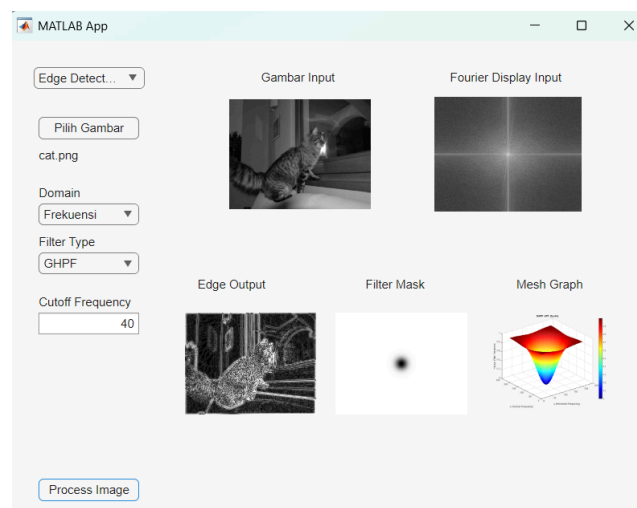
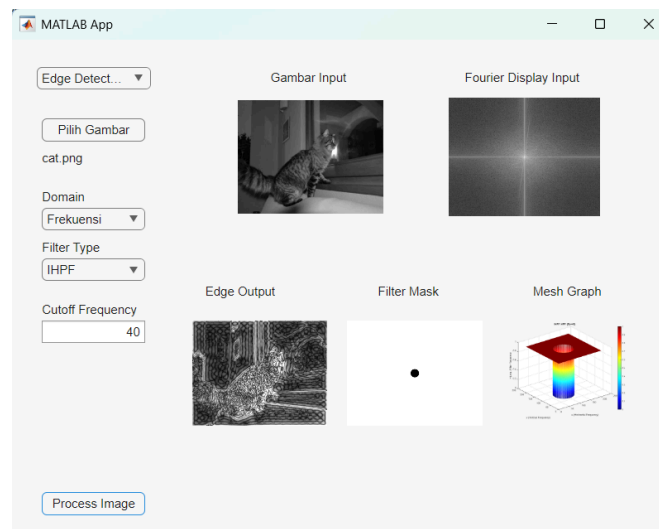
I. Gambar Grayscale



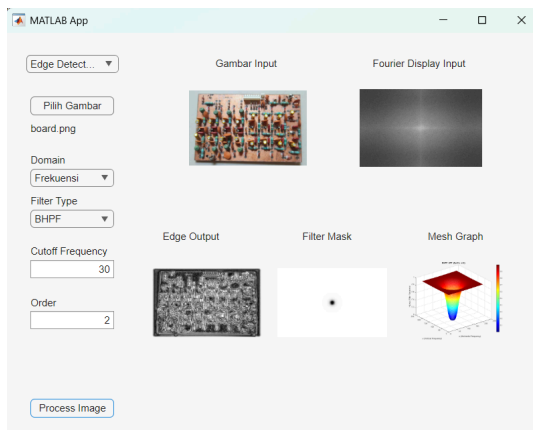
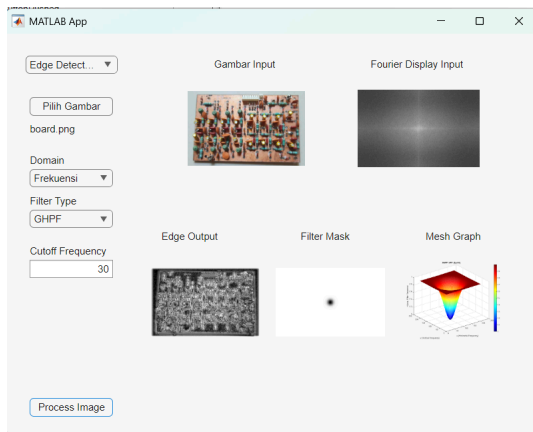
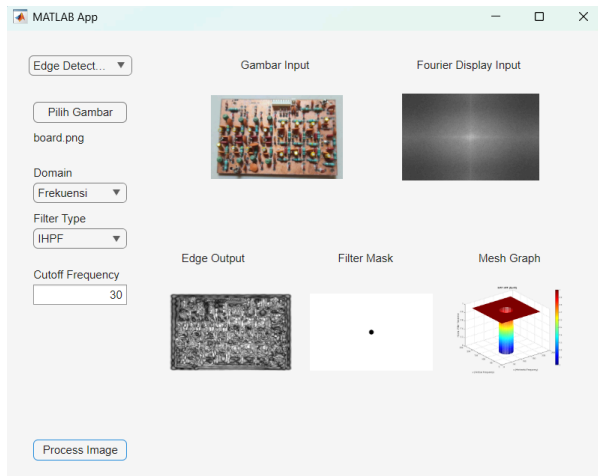
II. Gambar Berwarna



d. Edge Detector
I. Gambar Grayscale

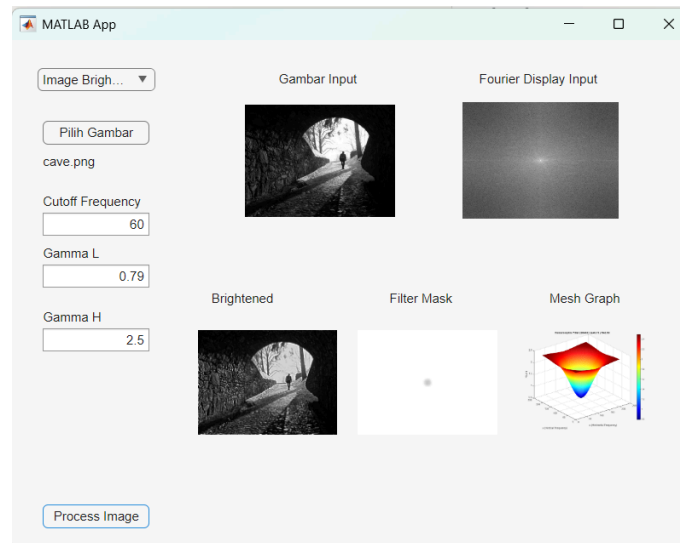


II. Gambar Berwarna

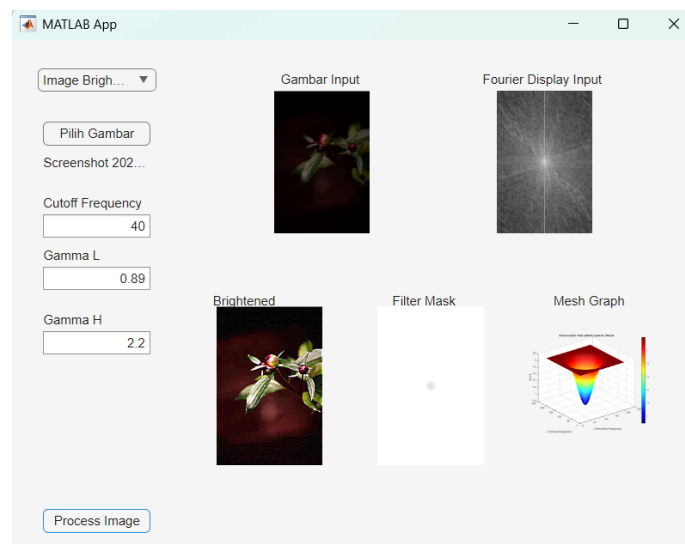


e. Image Brightness

I. Gambar Grayscale

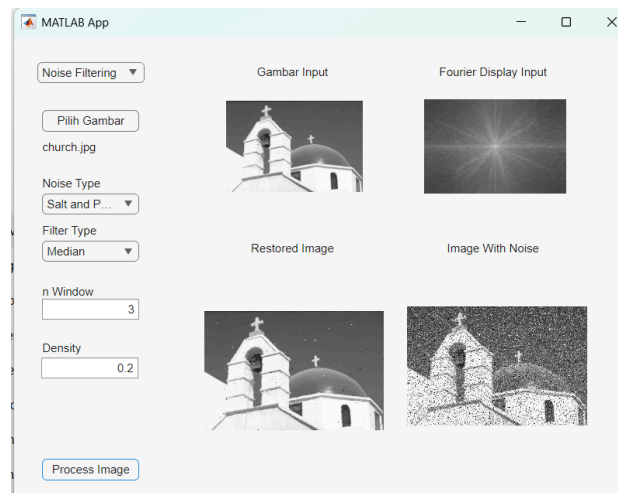
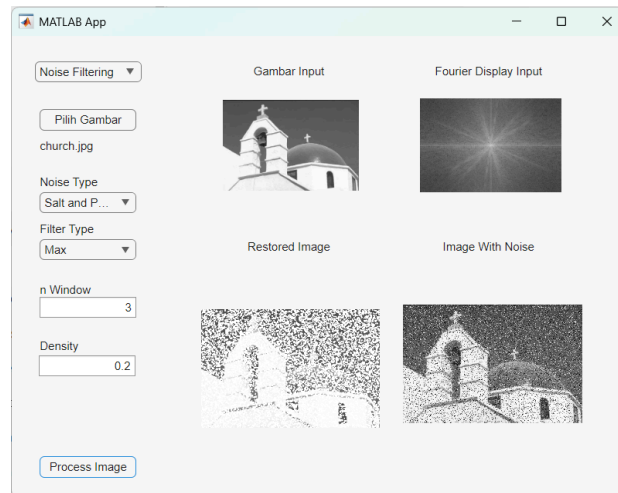
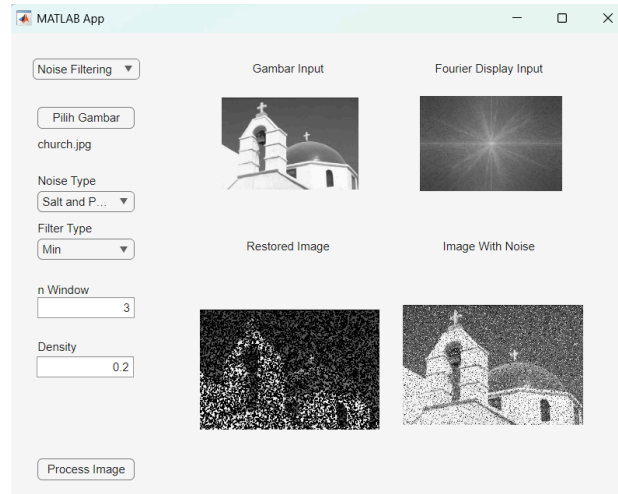


II. Gambar Berwarna

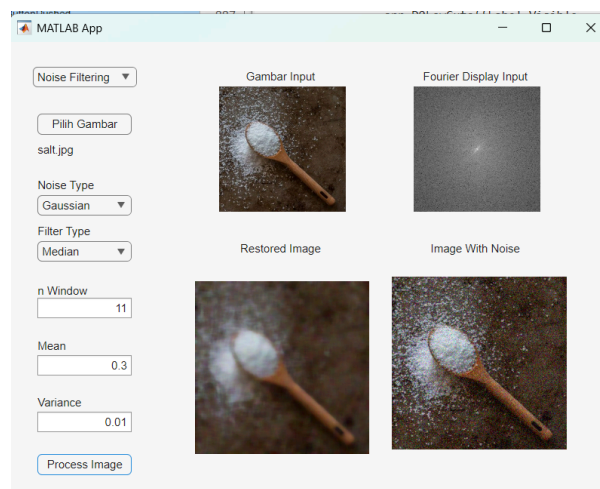
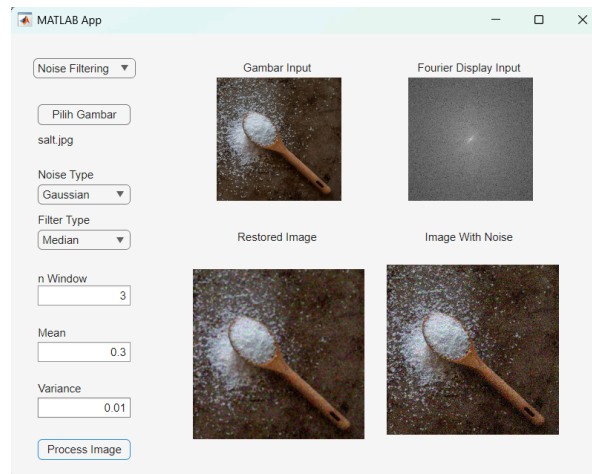
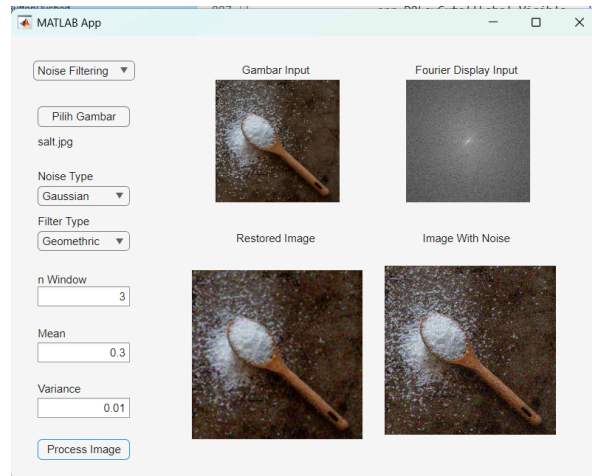


f. Noise Filtering

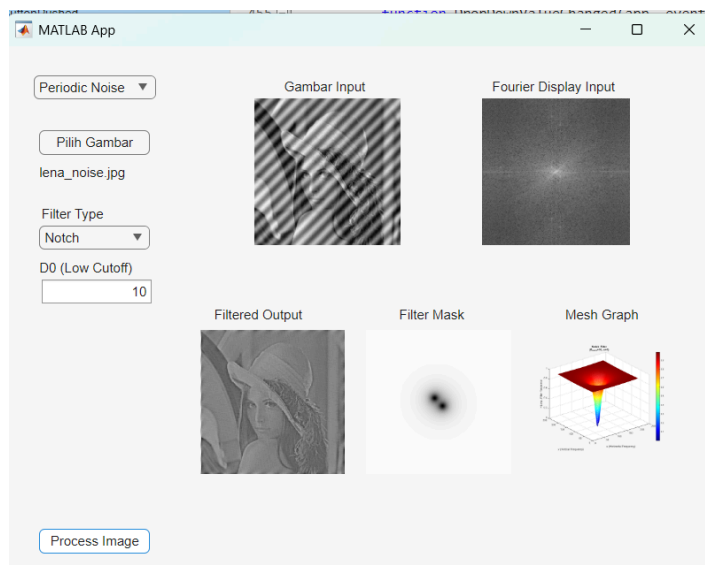
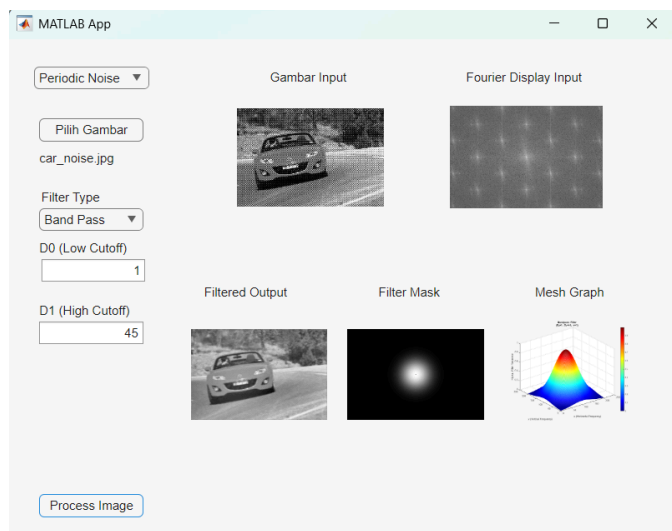
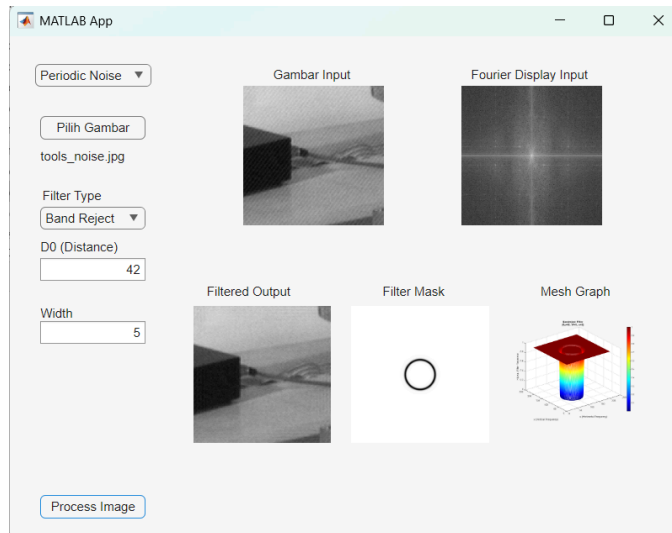
I. Gambar Grayscale

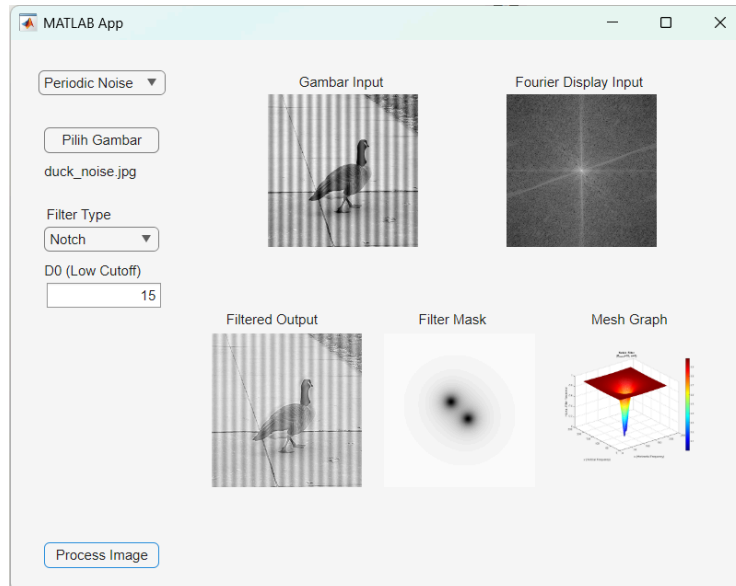
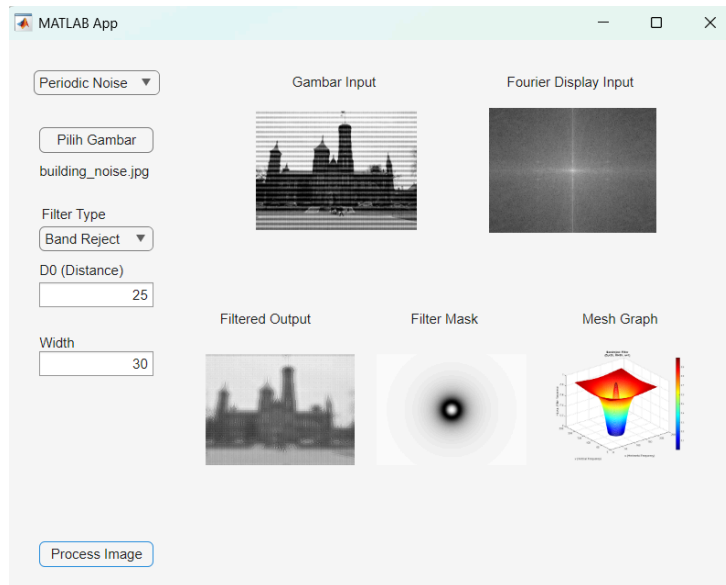


II. Gambar Berwarna

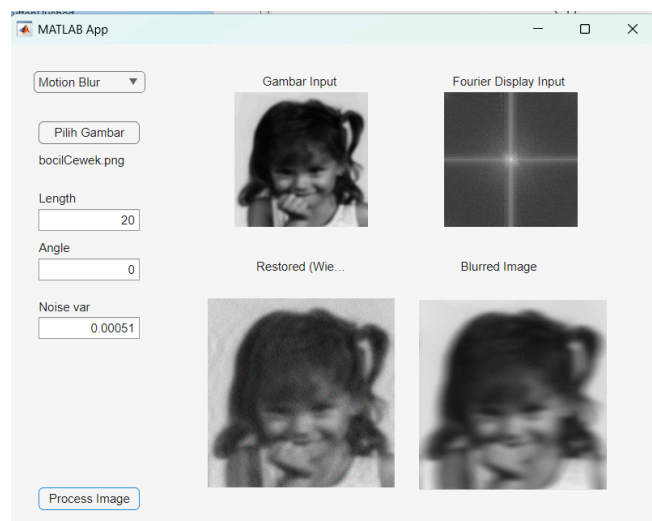
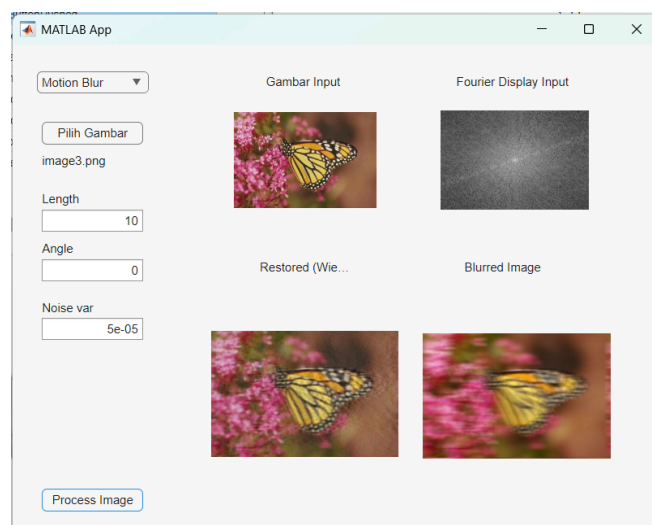
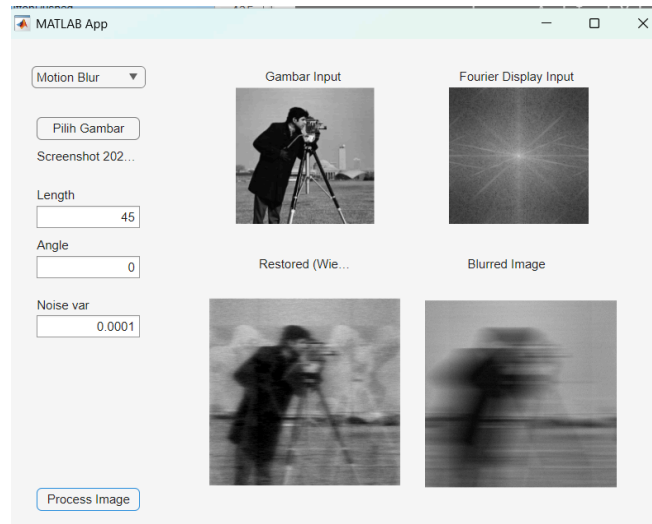


g. Periodic Noise





h. Motion Blur



BAB IV

ANALISIS KODE

A. Analisis cara kerja program untuk proses konvolusi citra

Berikut merupakan langkah-langkah yang dilakukan dalam proses konvolusi citra:

1. Sistem menerima input sebuah citra dari pengguna dan sebuah mask yang merupakan matriks $n \times n$. Mask ini digunakan untuk proses konvolusi kelak. Mask dapat memiliki tujuan berbeda-beda tergantung pada nilai-nilai yang dikandung didalamnya.
2. Sistem melakukan analisis pembacaan mask sehingga didapatkan nilai n_mask .
3. Sistem melakukan iterasi untuk setiap baris pada citra dan setiap kolom pada setiap baris yang diiterasi.
4. Nilai-nilai pinggir pada matriks diubah menjadi nilai awal matriks (bagian samping tidak dikonvolusi).
5. Nilai matriks yang tidak dianggap pinggir dilakukan konvolusi dengan pembacaan local matrix sebesar $n \times n$.
6. Local matrix dilakukan perhitungan dot product dengan mask yang ada. Hasil dari dot product ini menjadi hasil bagi citra baru yang dihasilkan pada pixel tersebut.
7. Setelah semua baris dan kolom selesai diiterasi, sistem mengembalikan citra baru sebagai output.

B. Analisis cara kerja program untuk proses Image Smoothing pada ranah spasial

Berikut merupakan langkah-langkah yang dilakukan dalam proses Image Smoothing pada ranah spasial:

1. Program menerima input dari pengguna berupa gambar, tipe spasial antara Mean atau Gaussian Filtering, banyaknya iterasi mask, dan juga menerima nilai sigma khusus untuk Gaussian Filtering. Terdapat pengecekan juga bahwa untuk nilai iterasi mask harus bernilai ganjil (1,3,5, dst) dan akan error jika bernilai genap.
2. Setelah menerima berbagai parameter tersebut, program akan mengecek input tipe spasial untuk pembentukan matriks. Terdapat dua kondisi dari pembentukan matriks yaitu sebagai berikut:
 - a. Matriks dari Mean Filtering Proses pembentukan matriks yang ada pada Mean Filtering adalah sebagai berikut:
 - I. Pembentukan ukuran matriks berdasar pada input $nMask$ diawal. Misalkan input sebesar tiga, maka matriks yang terbentuk akan berukuran 3×3 dengan seluruh nilai matriks adalah 1
 - II. Perhitungan total jumlah semua elemen yang ada pada matriks, misal jika matriks 3×3 , maka total nilainya adalah $1 \times 3 \times 3$ yaitu 9, jika matriks 5×5 , maka total nilainya adalah $1 \times 5 \times 5$ yaitu 25
 - III. Setelahnya, akan dilakukan pembagian terhadap setiap nilai dalam matriks dengan perhitungan total jumlah semua elemen

b. Matriks dari Gaussian Filtering

- I. Pembentukan ukuran matriks berdasar pada input nMask diawal. Misalkan input sebesar tiga, maka matriks yang terbentuk akan berukuran 3x3
- II. Melakukan pembentukan nilai pada matriks Gaussian dengan menentukan titik tengah sebagai koordinat (0,0) dan nilai ditentukan dengan menggunakan rumus Gaussian sebagai berikut

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- III. Setelahnya, akan dilakukan pembagian terhadap setiap nilai dalam matriks dengan perhitungan total jumlah semua elemen

C. Analisis cara kerja program untuk proses Image Smoothing pada ranah frekuensi dengan Low-Pass Filter, High-Pass Filter, dan penapisan untuk membuat citra lebih terang (Homomophic Filter)

Berikut merupakan langkah-langkah yang dilakukan dalam proses Image Smoothing pada ranah frekuensi:

1. Program menerima sejumlah input wajib dari pengguna yaitu gambar, jenis frekuensi dan nilai cutoff. Ada juga beberapa parameter yang bersifat opsional seperti n yang digunakan untuk Butterworth Filter serta gammaL dan gammaH yang digunakan untuk penerangan citra dengan memanfaatkan Homomorphic Filter
2. Setelah menerima input, akan dilakukan pendefinisian terhadap padding parameters yang pada program dibuat default untuk P bernilai 2*M dan Q bernilai 2*N dengan M dan N merupakan banyaknya pixel yang terdapat pada gambar. Dari padding parameters ini kemudian dilakukan pembentukan citra padding dengan menambahkan pixel-pixel bernilai nol pada f(x, y).
3. Berikutnya kemudian menjalankan fungsi Fourier Transform pada padded image. Dalam program, akan dipanggil fungsi `fourier_transform` untuk menjalankan transformasinya. Untuk detail pada fungsi `fourier_transform` sendiri adalah sebagai berikut:
 - a. Proses transformasi akan dibedakan berdasarkan kondisi warna pada gambar apakah grayscale ataupun RGB berwarna
 - b. Untuk gambar grayscale, prosesnya yaitu melakukan transformasi dengan kode berikut `fftshift(fft2(pad))` dan setelahnya dilakukan perhitungan magnitude spectrum untuk menampilkan kondisi dari Transformasi Fourier dengan rumus $fourierDisplay = \log(1 + abs(fourierSpectrum))$

- c. Untuk gambar RGB berwarna, prosesnya kurang lebih sama seperti dengan grayscale, hanya saja untuk transformasi dan perhitungan magnitude dilakukan untuk ketiga jenis warna yaitu Red, Green, dan Blue
4. Setelah melakukan transformasi Fourier, kemudian memanggil fungsi penapis yaitu `filter_processing` berdasarkan kondisi frekuensi yang dipilih. Untuk detail pada fungsi `filter_processing` sendiri adalah sebagai berikut:
 - a. Designing filter dan membuat meshgrid untuk perhitungan jarak dari center dengan memanfaatkan rumus Euclidean
 - b. Melakukan set mesh tergantung pada tipe frekuensi masukan. Pada program, terdapat tujuh kondisi untuk tipe frekuensi antara lain sebagai berikut:

Keterangan: $D = \text{euclidean}$, $D0 = \text{cutoffFrequency}$

- I. Ideal Low-Pass Filter Rumus dari penerapan filter ini adalah sebagai berikut:

$$\text{mesh} = \text{double}(\text{euclidean} \leq \text{cutoffFrequency})$$

- II. Gaussian Low-Pass Filter Rumus dari penerapan filter ini adalah sebagai berikut:

$$\text{mesh} = e^{\left(-\frac{\text{euclidean}^2}{2 * \text{cutoffFrequency}^2}\right)}$$

- III. Butterworth Low-Pass Filter Rumus dari penerapan filter ini adalah sebagai berikut:

$$\text{mesh} = \frac{1}{\left(1 + \left(\frac{\text{euclidean}}{\text{cutoffFrequency}}\right)^{2n}\right)}$$

- IV. Ideal High-Pass Filter Rumus dari penerapan filter ini adalah sebagai berikut:

$$\text{mesh} = \text{double}(\text{euclidean} > \text{cutoffFrequency})$$

- V. Gaussian High-Pass Filter Rumus dari penerapan filter ini adalah sebagai berikut:

$$\text{mesh} = 1 - e^{\left(-\frac{\text{euclidean}^2}{2 * \text{cutoffFrequency}^2}\right)}$$

- VI. Butterworth High-Pass Filter Rumus dari penerapan filter ini adalah sebagai berikut:

$$\text{mesh} = \frac{1}{\left(1 + \left(\frac{\text{cutoffFrequency}}{\text{euclidean}}\right)^{2n}\right)}$$

VII. Homomorphic Filter Rumus dari penerapan filter ini adalah sebagai berikut:

$$mesh = (\gamma_H - \gamma_L) * mesh_{GHPF} + \gamma_L$$

5. Selanjutnya yaitu mencari nilai konvolusi dengan ranah frekuensi menggunakan rumus berikut $convResult = meshArrays .* fourierSpectrum$ dan perkalian dilakukan berdasarkan kondisi gambar, jika grayscale akan dilakukan perkalian untuk satu matriks saja, tetapi jika gambar adalah RGB berwarna, iterasi perkalian akan dilakukan untuk setiap jenis warna
6. Mengambil bagian real dengan menjalankan inverse Fourier Transform untuk mendapatkan filtered image. Kondisi juga sama bahwa inverse untuk gambar grayscale akan dilakukan satu iterasi saja dan untuk gambar RGB berwarna akan dilakukan iterasi untuk setiap jenis warna
7. Memotong gambar pada bagian atas untuk mengembalikan pada semula

D. Analisis cara kerja program untuk proses penghilangan derau dengan Noise Filtering

Berikut merupakan langkah-langkah yang dilakukan dalam proses noise filtering pada citra:

1. Sistem menerima sebuah input berupa citra dari pengguna. Selain citra, sistem juga menerima tipe derau, yang dapat bernilai salt and pepper dan gaussian, dan tipe filter, yang merupakan algoritma dalam penyelesaian derau yang diberikan.
2. Selanjutnya, sistem memberikan derau/ noise ke dalam citra berdasarkan jenis tipe derau yang dimasukkan oleh pengguna. Pengguna juga dapat melakukan konfigurasi terkait density, nilai mean, dan nilai variance yang dapat mempengaruhi hasil derau.
3. Setelah dihasilkan citra dengan derau, sistem akan melakukan penyelesaian untuk menghilangkan derau yang telah ditambahkan kedalam citra tersebut.
4. Sistem melakukan iterasi untuk setiap baris dan setiap kolom untuk setiap barisnya.
5. Untuk pixel yang bukan merupakan pixel pinggir pada citra, dilakukan pencarian local matrix.
6. Sistem mengganti nilai pixel tersebut dengan hasil perhitungan dari local matrix. Hasil perhitungan ini bergantung pada tipe filter yang dimasukkan oleh pengguna. Filter-filter tersebut antara lain:
 - a. Min, mencari nilai minimum,

- b. Max, mencari nilai maksimum,
- c. Median, mencari nilai tengah,
- d. Arithmetic mean, mencari nilai rerata aritmatika,
- e. Geometric mean, mencari nilai rerata geometri,
- f. Harmonic mean, mencari nilai rerata harmonik,
- g. Contraharmonic mean, mencari nilai rerata kontra harmonik,
- h. Midpoint, mencari nilai rerata dari nilai min dan max, dan
- i. Alpha-trimmed mean, mencari nilai rerata dengan membuang nilai ujung pada local matrix.

7. Setelah seluruh pixel dikalkulasikan dan sistem telah mengiterasi seluruh pixel yang ada, sistem akan mengembalikan citra baru yang dihasilkan sebagai output.

E. Analisis cara kerja program untuk proses penghilangan derau dengan Periodic Noise Restoration

Fungsi ini berfungsi untuk menyelesaikan permasalahan akibat derau. Akan tetapi, metode penyelesaian yang dilakukan oleh fungsi ini berada dalam domain frekuensi. Berikut merupakan langkah-langkah yang dilakukan dalam proses periodic noise restoration:

1. Sistem menerima input berupa citra dari pengguna. Selain itu, sistem juga menerima tipe filter yang akan digunakan dan juga konfigurasi dari filter-filter tersebut.
2. Sistem melakukan transformasi citra tersebut menggunakan transformasi fourier agar dihasilkan tampilan citra tersebut dalam domain frekuensi.
3. Pada tampilan hasil transformasi fourier, pengguna dapat melihat bagaimana pola yang dibentuk akibat derau pada citra.
4. Pengguna memasukkan konfigurasi berupa W (width), D0 (low cutoff), dan D1 (high cutoff) ke sistem agar penyelesaian derau dapat lebih optimal dilakukan.
5. Sistem melakukan penyelesaian derau tersebut dengan menggunakan tipe filter yang dimasukkan oleh pengguna. Tipe filter tersebut antara lain adalah band-reject, band-pass, dan notch.
6. Sistem menghasilkan sebuah citra baru berdasarkan proses penggunaan filter tersebut dan mengembalikan citra tersebut sebagai output

F. Analisis cara kerja program untuk proses penghilangan Motion Blur dengan Penapis Wiener

1. Program menerima sejumlah input wajib dari pengguna yaitu gambar, nilai len dan theta untuk pemrosesan penapis, dan noise_var untuk ukuran derau aditif
2. Setelahnya, gambar akan dibuat blur terlebih dahulu dengan menjalankan fungsi motion blur untuk proses blur. Dari program kami, proses blurring dibuat secara circular untuk defaultnya
3. Setelah terbentuk gambar blur, program kemudian memanggil fungsi untuk menjalankan penapisan Wiener secara scratch dengan perhitungan formula

Wiener untuk setiap channel. Kondisi dibedakan berdasarkan gambar jika grayscale, iterasi akan dilakukan sekali saja dan untuk gambar RGB berwarna, akan dilakukan iterasi untuk tiap jenis warna

4. Untuk proses perhitungan formula Wiener dilakukan dengan memanggil fungsi `wiener_calculation` dengan proses sebagai berikut:
 - a. Menjalankan Fourier Transform untuk image dan blur kernel secara terpisah. Image kernel untuk $G(u,v)$ dan Blur kernel untuk $H(u,v)$
 - b. Menjalankan padding dan circular shift untuk memastikan ukuran gambar tetap sama setelah proses transformasi Fourier
 - c. Menghitung penapis Wiener dengan formula sebagai berikut

$$H_w(u, v) = \frac{1}{H(u, v)} \frac{|H(u, v)|^2}{|H(u, v)|^2 + noiseVar}$$

- d. Mendapatkan nilai citra restorasi dengan mengalikan penapis tersebut dengan citra degradasi dengan rumus $F(u, v) = H_w(u, v) * G(u, v)$
- e. Scale gambar ke dalam bentuk gray menggunakan fungsi `mat2gray` untuk mendapatkan hasil akhir

LAMPIRAN

Link github: https://github.com/Haikalin/IF4073_Tugas2